

OOP

UNIT 12 – THE STANDARD TEMPLATE LIBRARY

Knowing how a container works helps us implement one. Knowing which container to choose helps us design software.

LEARNING OBJECTIVES

By the end of this unit, you should be able to:

- Explain the purpose of the C++ Standard Library and its connection to generic programming.
- Distinguish between containers, iterators, and algorithms.
- Compare the major container families and choose an appropriate container for a problem.
- Use standard containers and iterators, including `begin()`, `end()`, and half-open ranges.
- Use selected standard algorithms and explain how Standard Library types support resource-safe design.

FROM IMPLEMENTING CONTAINERS TO CHOOSING CONTAINERS

Throughout this course, we repeatedly built classes that store and manage collections of objects. We allocated memory, resized arrays, copied objects, released resources, wrote templates, and implemented iterators.

Those implementations were important because they showed us how containers work. In professional C++, however, we usually do not build a new dynamic array, linked list, set, or map every time a program needs a collection.

C++ already provides reusable generic containers as part of the Standard Library. The new question is no longer only “How can we implement this data structure?” It is also “Which existing data structure best represents this problem?”

WHY DO WE NEED THE STANDARD LIBRARY?

Consider the Playlist class we used earlier in the course. A manually managed dynamic array might look like this:

```
class Playlist
{
private:
    Song* songs;
    int size;
    int capacity;

    // allocation, copying and cleanup
    // are our responsibility
};
```

This design taught us dynamic memory allocation, capacity management, copying, destruction, and the Rule of Three. But a music application is not special in needing a growing collection. Universities store students, stores keep products, banks keep transactions, and games keep players. The Standard Library provides reusable components so application classes can concentrate on the problem they model.

THE STANDARD LIBRARY AND GENERIC PROGRAMMING

The containers in the Standard Library are generic. This should now look familiar because we have already studied function templates and class templates.

```
template <typename T>
class Box
{
    T value;
public:
    void set(const T& value);
    T get() const;
};
```

The same class template can be instantiated with different types:

```
Box<int> numbers;
Box<std::string> text;
Box<Song> songBox;
```

Standard containers use the same idea:

```
std::vector<int> numbers;
std::vector<std::string> names;
std::vector<Song> songs;
```

THREE IMPORTANT IDEAS: CONTAINERS, ITERATORS, AND ALGORITHMS

A useful first model of STL-style programming contains three cooperating ideas.

Idea	Responsibility	Examples
Containers	Store objects	vector, list, set, map
Iterators	Represent positions and ranges	begin(), end()
Algorithms	Perform reusable operations on ranges	find(), count(), sort()

For example, `std::vector<Song>` is a container. `songs.begin()` and `songs.end()` produce iterators. `std::find` can use those iterators to search the range.

```
std::vector<Song> songs;  
  
auto it = std::find(songs.begin(), songs.end(), target);
```

DESIGN INSIGHT

The algorithm does not need to know the internal representation of the container. It works through a common iterator interface.

CHOOSING A CONTAINER STARTS WITH THE PROBLEM

Before selecting a container, ask what the program actually needs. The important questions are about behavior and guarantees, not about which class name is easiest to remember.

- Does the order of elements matter?
- Do I need access by position or index?
- Will I frequently add or remove elements at the front or back?
- Will I insert or remove elements through positions in the middle?
- Do I need to search using a key?
- Must values be unique?
- Should values remain sorted?
- Do I need FIFO, LIFO, or priority behavior?

A container should be chosen because its behavior matches the problem. It should not be chosen simply because it is the only container we remember.

SEQUENCE CONTAINERS

A sequence container stores elements in a sequence. The order of the elements is meaningful. Three important examples are vector, deque, and list.

```
std::vector<Song> songs;  
std::deque<Song> history;  
std::list<Song> linkedSongs;
```

All three store sequences, but they make different trade-offs. The correct choice depends on the operations the application performs most often.

Requirement	vector	deque	list
Indexed access	Yes	Yes	No
Efficient add at end	Yes	Yes	Yes
Efficient add at front	Usually not preferred	Yes	Yes
Internal insertion/removal	May move elements	May move elements	Natural through an iterator
Contiguous storage	Yes	No single block guaranteed	No

STD::VECTOR

A vector is a dynamically sized sequence. It can grow as elements are inserted, and it supports direct indexed access.

```
#include <vector>

std::vector<Song> songs;

songs.push_back(song1);
songs.push_back(song2);
songs.push_back(song3);

cout << songs.size() << endl;
songs[0].print();
```

A vector is a natural choice when sequence order matters, indexed access is useful, and most additions happen near the end.

For many ordinary sequences, vector is an excellent default. It is not, however, the answer to every collection problem.

SIZE AND CAPACITY?

Earlier, our own dynamic-array class required members such as:

```
Song* songs;
int size;
```



```
int capacity;
```

A vector manages these ideas internally. We can still inspect them:

```
cout << songs.size() << endl;  
cout << songs.capacity() << endl;
```

`size()` tells us how many elements currently exist. `capacity()` tells us how much storage is currently available before additional allocation may be required.

The important difference is responsibility. With our own array, our class had to allocate, resize, copy, and release storage. With vector, the vector performs those responsibilities.

DESIGN INSIGHT

This is encapsulation applied to a container: the user asks for a growing sequence without managing the storage mechanism directly.

STD::DEQUE

Suppose an application frequently adds and removes elements from both ends of a sequence. A deque may be more appropriate.

```
#include <deque>

std::deque<Song> history;

history.push_back(song1);
history.push_front(song2);

history.pop_back();
history.pop_front();
```

A deque also supports indexed access:

```
history[0].print();
```

The design question is not “Is deque faster?” in the abstract. It is “Does this sequence naturally require efficient work at both ends?”

STD::LIST

A list represents a linked sequence. It does not support indexed access because linked nodes are reached by moving through the sequence.

```
#include <list>

std::list<Song> songs;
```

This is not valid:

```
songs[5];    // Error: list has no indexed access
```

Traversal is natural through iterators or a range-based for loop:

```
for (const Song& song : songs)
{
    song.print();
}
```

A list is useful when the program naturally works with iterator positions and insertion or removal through those positions is important, while random indexed access is not required.

WHEN THE PROBLEM IS NOT REALLY A SEQUENCE

Suppose the music application stores genres: Rock, Jazz, Classical, and Electronic. Do we care that Rock is element 0? Probably not. Do we want Rock to appear seventeen times? Probably not.

The real requirement is not “store a sequence.” It is “store a collection of unique values.” That leads us to a different family of containers.

DESIGN INSIGHT

A good data structure does more than hold values. It can express rules about those values. Choosing a set communicates uniqueness directly in the program design.

STD::SET

```
#include <set>

std::set<std::string> genres;

genres.insert("Rock");
genres.insert("Jazz");
genres.insert("Classical");
genres.insert("Rock");
```

The second insertion of "Rock" does not create another copy. Uniqueness is part of the container's behavior.

A set also maintains its values in sorted order. Iterating through the set of strings will therefore follow string ordering.

```
for (const std::string& genre : genres)
{
    cout << genre << endl;
}
```

Use set when both uniqueness and ordered traversal are meaningful requirements.

STD::UNORDERED_SET

An unordered_set also stores unique values, but it does not maintain them in sorted order.

```
#include <unordered_set>

std::unordered_set<std::string> genres;

genres.insert("Rock");
genres.insert("Jazz");
genres.insert("Classical");
```

Do not write code that depends on the order in which an unordered container happens to display its elements.

A useful decision is therefore: if values must be unique, choose set when ordered traversal matters and consider unordered_set when ordering is irrelevant and lookup is central.

SEARCHING A SET

Suppose we want to know whether "Jazz" exists. Both set and unordered_set provide find().

```
auto it = genres.find("Jazz");

if (it != genres.end())
{
    cout << "Genre exists" << endl;
}
```

If the value exists, find() returns an iterator identifying it. If the value does not exist, the returned iterator equals end().

This pattern appears throughout the Standard Library and reinforces why iterators are an important common abstraction.

KEY-VALUE RELATIONSHIPS

Now consider a different requirement. Every song in the application has a unique ID:

```
101 -> Imagine  
102 -> Yesterday  
103 -> Halo
```

This is not naturally described as a sequence. The important relationship is key to value.

C++ provides map-like containers specifically for this situation.

STD::MAP

```
#include <map>  
std::map<int, Song> songs;  
songs[101] = imagine;  
songs[102] = yesterday;  
songs[101].print();
```

Here int is the key type and Song is the value type. The key has semantic meaning.

We are not saying “Give me the song at position 101.” We are saying “Give me the song whose ID is 101.”

A map keeps its keys ordered. It is therefore useful when both key-based lookup and ordered traversal of the keys matter.

WHY NOT JUST USE A VECTOR?

A vector could still store the songs. If each Song contains an ID, we could search sequentially:

```
for (const Song& song : songs)
{
    if (song.getId() == 5831)
    {
        // found
    }
}
```

That may be perfectly acceptable if sequential processing is what the program normally does.

But if the main operation is repeatedly “find the object associated with this key,” a map or unordered_map expresses the problem more directly.

DESIGN INSIGHT

Several containers may be able to hold the same data. The strongest choice is usually the one whose interface and guarantees best match the way the program uses that data.

STD::UNORDERED_MAP

```
#include <unordered_map>

std::unordered_map<int, Song> songsById;

songsById[101] = imagine;
songsById[102] = yesterday;
```

An unordered_map does not maintain its keys in sorted order. It is often a natural choice when lookup by key is important and sorted traversal of the keys is not required.

Requirement	Ordered container	Unordered container
Unique values	set	unordered_set
Key -> value	map	unordered_map
Sorted traversal	Yes	No
Conceptual implementation	Balanced tree	Hash table
Typical lookup	$O(\log n)$	$O(1)$ average

The complexity row is useful, but it should not replace the design question: Do I actually require ordering?

CONTAINER ADAPTERS

Some data structures are defined less by their physical representation and more by the operations they allow. C++ provides container adapters that expose specialized access behavior.

```
std::stack<Command> undoHistory;  
std::queue<Song> playQueue;  
std::priority_queue<Request> requests;
```

These types intentionally restrict their interfaces. That restriction makes the intended behavior clear.

STD::STACK – LAST IN, FIRST OUT

A stack follows LIFO: the most recently added element is the first one removed. Undo history is a natural example.

```
#include <stack>

std::stack<Command> undoHistory;

undoHistory.push(command);
Command last = undoHistory.top();
undoHistory.pop();
```

The interface deliberately does not provide arbitrary indexed access. A stack models a particular access discipline rather than a general-purpose sequence.

STD::QUEUE – FIRST IN, FIRST OUT

A queue follows FIFO: the first element inserted is the first one processed. A play queue is a natural example.

```
#include <queue>

std::queue<Song> playQueue;
```

```
playQueue.push(song);  
playQueue.front().play();  
playQueue.pop();
```

If the requirement says that songs should be played in the same order in which users requested them, queue communicates that behavior directly.

STD::PRIORITY_QUEUE – PRIORITY FIRST

Sometimes arrival order is not the important rule. A scheduler may need the highest-priority job first.

```
#include <queue>  
  
std::priority_queue<int> priorities;  
  
priorities.push(10);  
priorities.push(50);  
priorities.push(20);  
  
cout << priorities.top();    // 50
```

The important question is now “Which element currently has the highest priority?” rather than “Which element arrived first?”

THE CONTAINER DECISION TABLE

Problem requirement	Container to consider
Fixed-size array	<code>std::array</code>
General dynamically sized sequence	<code>std::vector</code>
Sequence with frequent work at both ends	<code>std::deque</code>
Linked sequence; iterator-based insertion/removal matters	<code>std::list</code>
Unique values in sorted order	<code>std::set</code>
Unique values; order unimportant	<code>std::unordered_set</code>
Key -> value with ordered keys	<code>std::map</code>
Key -> value; key order unimportant	<code>std::unordered_map</code>
LIFO behavior	<code>std::stack</code>
FIFO behavior	<code>std::queue</code>
Highest-priority element first	<code>std::priority_queue</code>

This table is a starting point, not a substitute for reasoning. Real programs may have several valid choices depending on which operations and guarantees are most important.

DESIGNING THE MUSIC APPLICATION WITH STL CONTAINERS

Different parts of the same application can naturally require different containers. There is no reason to force one container type onto every relationship.

Application need	Natural representation	Reason
Playlist	<code>vector<Song></code>	Ordered sequence; indexed access; append songs
Music catalog by ID	<code>unordered_map<int, Song></code>	Lookup by unique key; ordering unnecessary
Genres displayed alphabetically	<code>set<string></code>	Unique values plus sorted order
Play requests	<code>queue<Song></code>	FIFO behavior
Recent-history buffer	<code>deque<Song></code>	Efficient work at both ends
Undo commands	<code>stack<Command></code>	Latest command undone first

Choosing different containers for different responsibilities is not inconsistency. It is a sign that each part of the design reflects its own requirements.

ITERATORS REVISITED

The containers are different, yet C++ provides a common concept for moving through them: the iterator. An iterator behaves somewhat like a generalized pointer to a position in a container.

```
std::vector<Song> songs;

auto it = songs.begin();
it->print();
++it;
```

The same basic traversal pattern works with many containers even though their internal data structures differ.

```
for (auto it = songs.begin(); it != songs.end(); ++it)
{
    it->print();
}
```

BEGIN() & END()

Most standard containers provide `begin()` and `end()`. `begin()` identifies the first element. `end()` does not identify the last element; it represents one position after the last element.

```
begin() -> [Song][Song][Song]
                ^
                |
                end()
```

Therefore, dereferencing `end()` is invalid:

```
auto it = songs.end();
// it->print();    // WRONG: end() is not an element
```

`end()` is a boundary marker. Its one-past-the-end meaning makes range processing simple and consistent.

WHY AUTO IS USEFUL WITH ITERATORS

The full iterator type for a vector can be written explicitly:

```
std::vector<Song>::iterator it = songs.begin();
```

But the compiler already knows the exact type returned by `begin()`, so we can write:

```
auto it = songs.begin();
```

Using `auto` here does not remove type safety. It avoids repeating a long type that is already determined by the expression.

CONST ITERATORS

```
void printSongs(const std::vector<Song>& songs) {  
    for (auto it = songs.cbegin(); it != songs.cend(); ++it)    {  
        it->print();  
    }  
}
```

`cbegin()` and `cend()` provide read-only access through the iterator. This continues the course-wide principle that non-modifying intent should be expressed in the type system.

RANGE-BASED FOR

When the goal is simply to visit every element, a range-based for loop is often the clearest form.

```
for (const Song& song : songs) {  
    song.print();  
}
```

The explicit iterator has not disappeared conceptually. The loop uses the same range idea while hiding the traversal syntax.

Use the simplest form that communicates the intention clearly. Explicit iterators remain important when positions, erase operations, or iterator-based algorithms matter.

STANDARD ALGORITHMS

Containers solve the storage problem, but many operations on collections are also common: searching, counting, sorting, reversing, and transforming. The Standard Library provides generic algorithms for these repeated tasks.

Most algorithms operate on iterator ranges rather than on a particular container class.

```
#include <algorithm>

auto it = std::find(songs.begin(), songs.end(), target);
```

If the element is found, the returned iterator identifies it. If it is not found, the returned iterator equals `songs.end()`.

THE HALF-OPEN RANGE [FIRST, LAST)

Standard algorithms commonly use a half-open range written conceptually as `[first, last)`. The first position is included, while the last position is excluded.

```
std::find(songs.begin(), songs.end(), target);
```

This searches from the first element up to, but not including, `end()`. The convention works naturally with the one-past-the-end iterator.

STD::COUNT

```
std::vector<int> ratings{5, 3, 5, 4, 5};

int count = std::count(
    ratings.begin(),
    ratings.end(),
    5
);

cout << count;    // 3
```

STD::SORT – SEVERAL COURSE IDEAS WORKING TOGETHER

Suppose Song defines a meaningful comparison. A vector can then be sorted using the standard algorithm:

```
std::sort(songs.begin(), songs.end());
```

This one statement connects several topics from the course: classes, operator overloading, templates, iterators, and generic algorithms.

It demonstrates the purpose of common interfaces: independently designed components can work together when they satisfy shared expectations.

NOT EVERY ALGORITHM WORKS WITH EVERY CONTAINER

Consider a linked list:

```
std::list<Song> songs;
```

This is not valid:

```
std::sort(songs.begin(), songs.end());    // Error
```

`std::sort` requires iterator capabilities that a list does not provide. A list offers its own sorting operation:

```
songs.sort();
```

Iterators provide a common abstraction, but different containers still have different structural capabilities.

DESIGN INSIGHT

Generic programming does not mean every operation is valid for every type. A generic algorithm can only require operations that its types actually support.

STD::STRING REVISITED

We have used `std::string` since the beginning of the course. We can now recognize it as a sophisticated Standard Library class that manages a dynamically sized sequence of characters.

```
std::string title = "Imagine";  
std::string copy = title;  
copy = "Yesterday";  
  
std::string description = title + " - " + artist;
```

We never allocate a character array manually, implement deep copying, overload `+` ourselves, or call `delete[]` when a string goes out of scope.

`std::string` therefore illustrates many principles we studied throughout the course: encapsulation, constructors, destructors, copying, operator overloading, and resource management.

STANDARD CONTAINERS AND THE RULE OF ZERO

Earlier, a Playlist that directly owned dynamic memory :

```
class Playlist {  
    Song* songs;  
    int size;  
    int capacity;  
}; //
```

Copying and destruction require explicit attention.

Now consider:

```
class Playlist  
{  
private:  
    std::string name;  
    std::vector<Song> songs;  
};
```

Because string and vector manage their own resources, Playlist can often rely on compiler-generated special members. This is the Rule of Zero: prefer composing classes from resource-managing types so your own class does not manage resources directly.

CHOOSING BY REQUIREMENTS, NOT BY HABIT

Requirement	Strong candidate	Reason
Songs in playlist order; access song 5	<code>vector<Song></code>	Ordered sequence with direct indexed access
Unique usernames; order irrelevant	<code>unordered_set<string></code>	Uniqueness and membership lookup
Unique genres displayed alphabetically	<code>set<string></code>	Uniqueness plus sorted order
Retrieve song by unique ID	<code>unordered_map<int, Song></code>	Key-based lookup; order unnecessary
Playback requests in arrival order	<code>queue<Song></code>	FIFO semantics
Undo latest editing operation	<code>stack<Command></code>	LIFO semantics
Always process highest-priority request	<code>priority_queue<Request></code>	Priority semantics

Be able to explain the reason for the choice. Naming a container without relating it to the requirement is not enough.

PERFORMANCE IS PART OF THE DECISION

Container	Indexed access	Search	Add at end	Add at front
Vector	$O(1)$	$O(n)$	$O(1)$ amortized	$O(n)$
Deque	$O(1)$	$O(n)$	$O(1)$	$O(1)$
List	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Set	—	$O(\log n)$	$O(\log n)$	—
unordered_set	—	$O(1)$ average	$O(1)$ average	—
Map	By key	$O(\log n)$	$O(\log n)$	—
unordered_map	By key	$O(1)$ average	$O(1)$ average	—

Complexity is useful, but it should not be used mechanically. For example, choosing unordered_map simply because average $O(1)$ lookup is faster than $O(\log n)$ ignores whether sorted key order is a requirement.

Correct software design begins with semantics and operations, then considers performance.

A PRACTICAL DECISION PROCESS

What relationship do the elements have?

|
v

Is it fundamentally a sequence?

/

yes

|

Need indexed access?

/

yes

|

vector/
deque

\

no

|

list?

\

no

|

Unique values?

/

yes

|

set /
unordered
set

\

key -> value?

|

map /
unordered
map

Special access rule?

LIFO -> stack

FIFO -> queue

priority first -> priority_queue

No flowchart can make every design decision automatically. Its purpose is to force us to ask the right questions before choosing a type.

SOME BEGINNER MISTAKES

VECTOR IS NOT FOR EVERYTHING

Because vector is simple and powerful, it is easy to use it for every collection. But a collection of unique usernames used mainly for membership testing may be expressed more directly as an `unordered_set`.

```
std::vector<std::string> usernames;           // possible
std::unordered_set<std::string> usernames; // expresses uniqueness
```

CHOOSING ONLY BY COMPLEXITY

`unordered_map` is not automatically better than `map`. If sorted keys are required, `map` provides an important guarantee that `unordered_map` does not.

EXPECTING ORDER FROM AN UNORDERED CONTAINER

Do not write code whose correctness depends on the traversal order of `unordered_set` or `unordered_map`.

DEREFERENCING `END()`

```
auto it = songs.end();
// cout << *it;    // WRONG
```

`end()` is a boundary marker, not a valid element.

RESULT OF FIND()

```
auto it = songs.find(id);

if (it != songs.end())
{
    // safe to use the found element
}
```

COPYING ELEMENTS UNNECESSARILY

```
for (Song song : songs)           // copies each Song
{
    song.print();
}

for (const Song& song : songs)     // no copy, read-only access
{
    song.print();
}
```

THINKING STL REPLACES DATA-STRUCTURE KNOWLEDGE

Using map does not make trees irrelevant. Using unordered_map does not make hashing irrelevant. Using vector does not make dynamic arrays irrelevant.

Understanding the underlying data structures is what allows us to select the right abstraction. The Standard Library saves implementation effort; it does not eliminate algorithmic reasoning.

THE BIG PICTURE

The Standard Library should not feel like a completely separate subject at the end of an Object-Oriented Programming course. It is a practical application of many ideas we have spent the semester learning.

```
class
  -> encapsulation
  -> constructors and destructors
  -> copying and resource management
  -> relationships between classes
  -> operator overloading
  -> inheritance and polymorphism
  -> templates
  -> iterators
  -> reusable Standard Library components
```

A standard container is a class, usually a class template, responsible for its resources, traversable through iterators, and designed to work with generic algorithms.

SUMMARY

In this unit, we moved from implementing containers to selecting containers. Earlier implementations helped us understand memory management, copying, object lifetime, generic programming, and iterators. The Standard Library lets us reuse professionally implemented generic components instead of rebuilding these mechanisms for every application.

Sequence containers such as vector, deque, and list represent ordered collections with different access and insertion characteristics. Sets represent unique values. Maps represent key-value relationships. Container adapters represent specialized access policies such as LIFO, FIFO, and priority-first processing.

Iterators provide a common way to represent positions and ranges, while standard algorithms perform reusable operations on those ranges.

Most importantly, choosing a container is a design decision. There is no universally best container. The correct question is: What relationships, operations, and guarantees does this problem require?

REVIEW QUESTIONS

1. What problem does the C++ Standard Library solve for programmers?
2. Why are Standard Library containers an example of generic programming?
3. What are the roles of containers, iterators, and algorithms?
4. What characteristics make vector a good choice for many sequences?
5. Give a situation in which deque may be more appropriate than vector.
6. Why does list not provide the [] operator?
7. What requirement does set express naturally?
8. What is the main conceptual difference between set and unordered_set?
9. What relationship does map model?
10. When might unordered_map be preferable to map?
11. When might map still be preferable even if unordered_map offers $O(1)$ average lookup?
12. What behavior do stack, queue, and priority_queue provide?

REVIEW QUESTIONS – CONTINUED

13. What does `begin()` represent? What does `end()` represent?
14. Why must `end()` never be dereferenced?
15. Why can an iterator be described as a generalized pointer?
16. What does the half-open range `[first, last)` mean?
17. Why can `std::sort` be used with vector iterators but not directly with list iterators?
18. Why is `for (const Song& song : songs)` often preferable to `for (Song song : songs)`?
19. How does using `std::vector` inside `Playlist` reduce `Playlist`'s resource-management responsibilities?
20. What is the Rule of Zero?
21. Why is choosing a container primarily a software-design decision?
22. Choose a suitable container for each case and justify it: songs in playlist order; unique usernames; songs retrieved by ID; requests processed in arrival order; undo history; highest-priority jobs; unique genres displayed alphabetically.

LOOKING AHEAD

This unit completes our introduction to the major ideas of Object-Oriented Programming in C++. We began by asking why classes are needed if struct already exists. From there we learned how objects protect their state, how they are created and destroyed, how they manage resources, how classes relate to one another, how operators can be extended to user-defined types, how inheritance and polymorphism support flexible designs, and how templates make code generic.

In this unit, many of those ideas came together in the Standard Library. We are now able not only to implement classes and data structures, but also to make informed decisions about which abstractions should be used for a particular problem.

Writing correct C++ requires knowledge of syntax. Designing good C++ software requires understanding the consequences of our choices.

In Unit 13, we will bring the course together through exam preparation. Rather than introducing another major language feature, we will revisit relationships between the topics studied throughout the semester, trace code involving objects and inheritance, identify design and memory-management errors, and practice choosing appropriate C++ mechanisms for different problems.